

Министерство образования и науки Российской Федерации
Пермский институт (филиал)
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Сопроводительная записка
по дисциплине «Поддержка и тестирование программных
модулей»
тема: «Разработка WPF-приложения для управления складом с
покрытием
модульными тестами»

Выполнила:
студентка группы ИПо-42
Яснова Маргарита Витальевна

г. Пермь, 2026

Оглавление

Введение и анализ задания.....	3
Выбор архитектуры и технологий.....	5
Проектирование модели данных.....	6
Разработка сервисного слоя.....	8
1. Структура сервисов и их интерфейсы.....	8
2. OrganizationService и WarehouseService.....	8
3. ProductService и управление остатками.....	9
4. InvoiceService и логика накладных.....	9
Генерация тестовых данных.....	11
Разработка пользовательского интерфейса.....	12
Разработка модульных тестов.....	14
1. OrganizationServiceTests и WarehouseServiceTests.....	14
2. ProductServiceTests.....	14
3. InvoiceServiceTests.....	15
Трудности и способы их решения.....	17
Вывод.....	19

Введение и анализ задания

Тема практической работы была сформулирована в виде технического задания на разработку WPF-приложения «Управление складом». Задание предполагало создание системы учёта юридических лиц, складов, номенклатуры товаров и движения товаров посредством накладных. В отличие от учебных задач, где предметная область выдумывается самим студентом, здесь все требования были заданы заранее — и моей задачей было не придумать, а внимательно интерпретировать их и реализовать в полном объёме.

Анализ технического задания выявил несколько ключевых моментов, определивших дальнейшую архитектуру. Во-первых, система должна быть трёхуровневой: организация → склад → товар. Это не просто навигация между экранами, а иерархия связей в данных: каждый склад обязательно принадлежит какой-то организации, каждый товар — конкретному складу. Соответственно, в модели данных должны присутствовать внешние ключи `OrganizationId` в складе и `WarehouseId` в товаре.

Во-вторых, в задании явно описана партионная логика движения товаров. Поступление и списание происходят партиями, каждое движение сопровождается накладной — входящей или исходящей. При этом остаток на складе меняется не в момент добавления строки в накладную, а только после её подтверждения. Это потребовало ввести в модель накладной два состояния (черновик и проведена) и явный метод проведения, который изменяет остатки.

В-третьих, задание содержало архитектурное требование: вся логика и модели должны находиться в отдельной библиотеке классов, а каждый сервис и каждый его метод — покрыты модульными тестами. Данные хранятся в коллекциях в памяти, без использования базы данных. Это требование сразу определило структуру решения — три отдельных проекта и отсутствие зависимостей от WPF в библиотеке данных.

Наконец, задание содержало требования к пользовательскому интерфейсу: единая тема оформления через словарь ресурсов, отображение текущего контекста (выбранная организация и склад), валидация ввода и подтверждение удаления через ввод названия. Всё это было учтено при проектировании интерфейса.

Выбор архитектуры и технологий

Технологический стек был частично определён заданием: язык C#, фреймворк WPF, рекомендованная архитектура MVVM. Мне оставалось принять решения по тем вопросам, которые задание не оговаривало жёстко.

Разделение решения на три проекта прямо следовало из требования изолировать логику от интерфейса. Я создала:

- WarehouseLibrary — библиотека классов с моделями и сервисами;
- WarehouseApp — WPF-приложение с интерфейсом;
- WarehouseLibrary.Tests — тестовый проект.

Такое разделение даёт конкретный практический выигрыш. Тестовый проект ссылается только на библиотеку данных и не имеет WPF-зависимостей, а значит, тесты запускаются без графической подсистемы и работают быстро. Кроме того, сам факт того, что логика не может обратиться к элементам интерфейса, дисциплинирует: разработчик вынужден держать бизнес-логику чистой.

Платформа .NET 8.0 выбрана как актуальная версия с полной поддержкой WPF. Для модульного тестирования использован фреймворк xUnit с атрибутами [Fact] и [Theory]. Каждый тестовый класс изолирован: в конструкторе класса создаются новые экземпляры сервисов, что гарантирует независимость тестов друг от друга.

Паттерн MVVM я воспринимала как практический инструмент, а не как формальность. В проекте он реализован через три составляющих: модели (в библиотеке данных), ViewModels (в WPF-проекте) и View (XAML-окна). Для команд использована собственная реализация ICommand — класс RelayCommand, принимающий делегат выполнения и предикат CanExecute. Базовый класс BaseViewModel реализует INotifyPropertyChanged, благодаря чему изменения свойств автоматически отражаются в интерфейсе.

Проектирование модели данных

Прежде чем писать сервисы, я спроектировала модели. Задание фактически диктовало их структуру — мне оставалось формализовать требования в классах. Все модели размещены в пространстве имён `WarehouseData.Models`.

Organization содержит идентификатор `OrgId` типа `BigInteger` и название `OrgName`. Идентификатор генерируется автоматически через статический счётчик `OrgCounter` при создании объекта. Это простая сущность — корень иерархии.

Warehouse содержит `WhId` (тоже `BigInteger`), название `WhName`, адрес `WhAddress` и внешний ключ `OrgId`, ссылающийся на организацию. Наличие явного внешнего ключа позволяет фильтровать склады по организации простым LINQ-запросом.

Product оказался наиболее богатой по составу полей моделью, поскольку задание требовало конкретный набор колонок в таблице товаров: фото, наименование, категория, производитель, поставщик, цена, остаток, скидка. В качестве первичного ключа использован строковый артикул (`Article`) — это осознанное архитектурное решение: артикул уникален и удобен для отображения, а его использование в качестве идентификатора позволяет избежать дополнительных поисков по числовому `Id` при проведении накладных.

Связанные сущности **Category**, **Manufacturer** и **Supplier** вынесены в отдельные классы с числовым `Id` и названием. В товаре хранятся как внешние ключи (`CategoryId`, `ManufacturerId`, `SupplierId`), так и навигационные свойства (`Category`, `Manufacturer`, `Supplier`) — это позволяет обращаться к названиям справочников напрямую, без дополнительных запросов.

Отдельного внимания потребовало проектирование модели накладной. Задание описывало два типа движения: приход и расход. Я реализовала строковое поле `InvoiceType` со значениями "In" и "Out", а также поле `Status` для отражения состояния накладной ("Черновик" или "Проведена").

Класс **Invoice** содержит:

- InvoiceId типа BigInteger;
- InvoiceNumber — читаемый номер вида НК-20260516-3;
- CreatedDate — дата создания;
- InvoiceType — тип движения;
- WarehouseId и WarehouseName — контекст склада;
- Status — состояние;
- Items — коллекция строк List<InvoiceItem>;
- вычисляемое свойство TotalAmount — общая сумма по накладной.

Класс **InvoiceItem** хранит артикул товара (ProductArticle), его название (ProductName), количество (Quantity) и цену на момент добавления в накладную (Price). Фиксация цены в строке — важное архитектурное решение: если цена товара в справочнике изменится, история уже проведённых накладных останется корректной.

Флаг состояния накладной (Status) — ключевое проектное решение. Накладная существует в двух состояниях: черновик и проведена. Только проведение вызывает изменение остатков на складе. Это прямо соответствует требованию задания: «после заполнения накладной и подтверждения перемещения товаров количество этих товаров на складе должно измениться».

Разработка сервисного слоя

1. Структура сервисов и их интерфейсы

Задание требовало создать сервисы для CRUD-операций и покрыть каждый метод тестами. Для удобства я определила для каждого сервиса парный интерфейс: `IOrganizationService`, `IWarehouseService`, `IProductService`, `InvoiceService`. Интерфейсы дают два практических выигрыша: во-первых, они фиксируют публичный контракт сервиса, и при написании тестов сразу понятно, какие методы должны быть проверены. Во-вторых, использование интерфейсов открывает дорогу к внедрению зависимостей и подмене реализаций в тестах.

Все сервисы хранят данные в приватных коллекциях `List<T>`. Сервисы создаются как обычные классы, в WPF-приложении экземпляры создаются один раз при старте и передаются через конструктор `MainViewModel` — это гарантирует, что все операции работают с одними и теми же данными в течение сессии.

2. `OrganizationService` и `WarehouseService`

`OrganizationService` при явном вызове метода `SeedTestData()` наполняет внутренний список организаций. Все методы работают с приватным `List<Organization>`. Метод `GetById(BigInteger id)` находит организацию через LINQ-выражение. Метод `Update` изменяет поле `OrgName` найденной организации и возвращает её же — это позволяет `ViewModel` корректно обновить элемент в `ObservableCollection`. Метод `Delete` удаляет организацию из списка и возвращает логический результат операции.

`WarehouseService` устроен аналогично, но имеет дополнительный метод `GetByOrganizationId(BigInteger organizationId)`, который возвращает только те склады, которые принадлежат конкретной организации. Именно этот метод лежит в основе навигации «Организация → Склады»: при выборе организации `ViewModel` вызывает метод и наполняет коллекцию `Warehouses`.

Метод `SeedTestData(List<Organization> organizations)` принимает список организаций и создаёт для первых двух по несколько складов.

Такое решение позволяет связывать склады с реально существующими организациями, а не с «выдуманными» идентификаторами.

3. ProductService и управление остатками

ProductService хранит товары, а также связанные справочники — категории, производителей и поставщиков. При генерации тестовых данных каждый товар получает заполненные навигационные свойства `Category`, `Manufacturer` и `Supplier`, что позволяет отображать их названия в таблице без дополнительных запросов.

Особого внимания заслуживает метод `UpdateStock(string article, int quantityChange)`. Он принимает изменение остатка со знаком: положительное значение увеличивает остаток (приходная накладная), отрицательное — уменьшает (расходная накладная). Метод вызывается из `InvoiceService` при проведении накладной.

Важная деталь: метод защищает остаток от ухода в минус — если после изменения значение становится отрицательным, оно приводится к нулю. Это дополнительная подстраховка на случай, если логика проверок в `InvoiceService` будет нарушена.

Разделение ответственности между `ProductService` и `InvoiceService` реализовано сознательно: управление остатком находится в одном сервисе, логика формирования и проведения накладной — в другом. Это соответствует принципу единственной ответственности.

4. InvoiceService и логика накладных

InvoiceService — наиболее сложный сервис в проекте. Он принимает `ProductService` в конструкторе и хранит список накладных в приватной коллекции.

Метод `Create(string type, BigInteger warehouseId, string warehouseName)` создаёт новый объект `Invoice`, присваивает ему уникальный номер через статический счётчик и возвращает его. Накладная при этом находится в состоянии черновика — её можно наполнять строками.

Метод `AddItem` добавляет строку в накладную. Если товар с таким артикулом уже присутствует в накладной, количество суммируется — это предотвращает дублирование строк при повторном добавлении одного и того же товара.

Метод `ProcessInvoice(BigInteger invoiceId, IProductService productService)` реализует ключевую бизнес-логику. Сначала он проверяет, что накладная существует и находится в состоянии черновика. Затем по каждой строке вычисляется изменение остатка: для приходной накладной — положительное, для расходной — отрицательное. Изменения передаются в `ProductService.UpdateStock`. После успешного проведения накладная помечается статусом «Проведена».

Такая логика — «всё или ничего» — исключает ситуацию частичного проведения накладной и обеспечивает целостность данных.

Генерация тестовых данных

Задание требовало создать методы, заполняющие коллекции тестовыми данными. В моей реализации это методы `SeedTestData`, размещённые прямо в сервисах — такой подход проще, чем отдельный класс-генератор, поскольку данные принадлежат именно сервису и создаются в его контексте.

`OrganizationService.SeedTestData()` создаёт три организации с реалистичными названиями: «ООО Рога и Копыта», «ЗАО СтройДом», «НА Холодец».

`WarehouseService.SeedTestData(organizations)` принимает список организаций и создаёт для первых двух несколько складов: «Склад №1 Рога и Копыта», «Склад №2 Рога и Копыта», «Склад СтройДом». У каждого склада заполнен адрес.

`ProductService.SeedTestData(warehouses)` принимает список складов и для каждого из них создаёт по два товара: «Молоток строительный» и «Гвозди 100мм» — с заполненными категориями («Инструменты», «Крепеж»), производителями и поставщиками. Артикул каждого товара формируется по шаблону ART-000001, ART-000002 и так далее — это обеспечивает уникальность и удобство визуального поиска.

Метод `SeedTestData` вызывается один раз при старте приложения в `MainViewModel.LoadData()`. Благодаря проверке `if (!_products.Any())` повторные вызовы не создают дубликатов.

Использование `SeedTestData` в тестах даёт изолированность: каждый раз при создании нового экземпляра сервиса данные генерируются заново, независимо от предыдущих тестов.

Разработка пользовательского интерфейса

Задание содержало конкретные требования к интерфейсу: унифицированная тема через ResourceDictionary на уровне Application, отображение текущего контекста, валидация ввода. Интерфейс разбит на главное окно MainWindow и диалоговые окна для создания и редактирования сущностей.

Главное окно разделено на три вертикальные панели:

- слева — список организаций с кнопками «Добавить», «Изменить», «Удалить»;
- в центре — список складов выбранной организации;
- справа — таблица товаров выбранного склада.

Между панелями организована последовательная навигация: выбор организации автоматически загружает её склады, выбор склада — товары этого склада. Текущий контекст отображается в заголовках панелей: «Орг: ООО Рога и Копыта», «Склад: Склад №1».

Диалог OrganizationDialog запрашивает наименование компании. **WarehouseDialog** — название и адрес склада. **ProductDialog** содержит все поля товара: наименование, категорию, производителя, поставщика, цену, количество, скидку, единицу измерения, описание и путь к фотографии. **DeleteConfirmationDialog** реализует требование задания о подтверждении удаления организации через ввод её названия — только при точном совпадении кнопка «Удалить» срабатывает.

Диалог InvoiceDialog позволяет выбрать тип накладной, добавить строки с товарами и провести накладную. Для расходной накладной реализована проверка, что вводимое количество не превышает остаток на складе.

Для команд WPF использован RelayCommand — собственная реализация ICommand, принимающая Action и предикат CanExecute. Это позволяет привязывать кнопки к методам ViewModel через {Binding Command} без code-behind. Предикат CanExecute автоматически блокирует кнопку «Изменить» и «Удалить», если ничего не выбрано в списке.

Таблица товаров реализована через DataGrid с колонками согласно требованиям задания: Фото, Наименование, Категория, Производитель, Поставщик, Цена, Остаток, Скидка.

Единая тема оформления описана в словарях ресурсов Theme.xaml и Styles.xaml, подключённых на уровне Application. Все цвета и стили элементов (кнопок, полей ввода, таблиц) описаны в одном месте, что обеспечивает согласованный внешний вид всех окон.

Разработка модульных тестов

Требование задания было конкретным: каждый сервис и каждый метод сервиса должны быть покрыты модульными тестами. Это не пожелание, а обязательное условие. Все тесты используют xUnit, структурированы по принципу AAA (Arrange–Act–Assert) и изолированы друг от друга: в конструкторе каждого тестового класса создаются новые экземпляры сервисов.

1. OrganizationServiceTests и WarehouseServiceTests

OrganizationServiceTests содержит тесты для всех публичных методов OrganizationService. SeedTestData_ShouldAddOrganizations проверяет, что после инициализации список не пуст и содержит ровно три записи. Create_ValidName_ShouldCreateWithBigIntegerId проверяет, что созданная организация имеет ненулевой OrgId. GetById_ExistingOrganization_ShouldReturnIt и GetById_NonExisting_ShouldReturnNull проверяют корректность поиска по идентификатору. Update_ShouldChangeName проверяет, что метод действительно изменяет название. Delete_ShouldRemoveOrganization и Delete_NonExisting_ShouldReturnFalse проверяют удаление и граничный случай.

WarehouseServiceTests аналогичен по структуре, но содержит дополнительный тест GetByIdOrganizationId_ShouldReturnOnlyOrgWarehouses. Он берёт первую организацию и проверяет, что все возвращённые склады принадлежат именно ей. Это важный тест, проверяющий корректность фильтрации, лежащей в основе навигации «Организация → Склады».

2. ProductServiceTests

ProductServiceTests охватывает все публичные методы ProductService, включая:

- SeedTestData_ShouldCreateProducts — проверяет генерацию тестовых товаров;
- Create_Product_ShouldHaveArticle — проверяет, что после создания товара у него заполнен артикул;

- `GetByArticle_ExistingProduct_ShouldReturnIt` — проверяет поиск по артикулу;
- `Update_ShouldChangeProductFields` — проверяет изменение полей товара;
- `Delete_ShouldRemoveProduct` — проверяет удаление;
- `UpdateStock_ShouldChangeQuantity` — ключевой тест, изменяющий остаток на +50 и проверяющий, что значение действительно увеличилось ровно на 50;
- `UpdateStock_ShouldNotGoNegative` — тест граничного условия: списание 20 единиц товара при остатке 10 приводит к остатку 0, а не к отрицательному значению;
- `UpdateStock_IncreaseQuantity_ShouldAddToStock` и `UpdateStock_DecreaseQuantity_ShouldSubtractFromStock` — проверяют работу с положительным и отрицательным изменением;
- `ImportFromCsv_ValidData_ShouldImportProducts` — проверяет импорт двух товаров из CSV-строки.

3. InvoiceServiceTests

InvoiceServiceTests — наиболее содержательный тестовый класс, проверяющий все ключевые сценарии работы с накладными.

`Create_ShouldCreateInvoice` проверяет, что созданная накладная не равна null, имеет правильный тип, Id склада и статус «Черновик».

`AddItem_ShouldAddToInvoice` проверяет, что после добавления строки в накладную список `Items` содержит ровно один элемент с правильным количеством, а вычисляемое свойство `TotalAmount` возвращает корректную сумму ($5 \times 100 = 500$).

`ProcessInvoice_ShouldChangeStatus` — ключевой тест для приходной накладной. Он создаёт товар с начальным остатком 10, формирует накладную прихода на 5 единиц, проводит её и проверяет три вещи: метод вернул true, статус накладной стал «Проведена», остаток товара увеличился до 15.

`ProcessOutcomeInvoice_ShouldDecreaseStock` аналогичен, но для расходной накладной: товар с остатком 100, накладная расхода на 30

единиц, после проведения остаток равен 70. Этот тест проверяет корректность отрицательного изменения остатка.

Отдельно проверяются граничные случаи: невозможность провести накладную дважды, невозможность списать больше, чем есть на складе.

Трудности и способы их решения

Первая трудность — обновление интерфейса после редактирования. Когда я реализовал редактирование организации и склада, изменения не отображались в списке, хотя в сервисе данные корректно обновлялись. Причина оказалась в специфике `ObservableCollection`: изменение свойства объекта не вызывает событие `CollectionChanged`, а только `PropertyChanged` самого объекта. Я решила это через полное обновление списка: `ViewModel` перезапрашивает данные из сервиса и заново наполняет коллекцию. Это менее эффективно, чем точечное обновление, но гарантирует корректность отображения.

Вторая трудность — привязки данных в WPF. Несколько раз свойство в `ViewModel` менялось, но интерфейс не обновлялся. Причина — отсутствие реализации `INotifyPropertyChanged`. Решение: базовый класс `BaseViewModel` с методом `OnPropertyChanged()`, от которого наследуют все `ViewModel`. Диагностика велась через окно `Output` в `Visual Studio`, куда WPF записывает ошибки привязок при отладке.

Третья трудность — неактивные кнопки редактирования и удаления товаров. Кнопки не подсвечивались, потому что `CanExecute` возвращал `false` — команды не знали, что выбранный товар изменился. Решение: привязка `SelectedItem` в `DataGrid` к свойству `SelectedProduct` и вызов `CommandManager.InvalidateRequerySuggested()` при изменении выбранного элемента. Это заставляет WPF заново проверить все предикаты `CanExecute`.

Четвёртая трудность — ошибка кодировки при чтении XAML. При копировании кода из ответов возникла ошибка «недопустимый знак для указанной кодировки». Причина — сохранение файла в кодировке, отличной от UTF-8. Решение: сохранение всех XAML-файлов через диалог «Сохранить с кодировкой» с явным выбором UTF-8 с сигнатурой.

Пятая трудность — ресурсы стилей не находились в диалоговых окнах. Ошибка «Не удастся найти ресурс с именем `SectionHeaderStyle`» возникала потому, что диалоговые окна не наследуют словарь ресурсов из `App.xaml` автоматически в момент парсинга XAML. Я отказалась от использования глобальных стилей в диалогах, задав все атрибуты

оформления локально — это упростило отладку и сделало каждое диалоговое окно независимым.

Шестая трудность — тестовые данные появлялись на новых складах. Изначально SeedTestData вызывался при каждом выборе склада, из-за чего товары «Молоток» и «Гвозди» появлялись в том числе на только что созданных пользователем складах. Решение: перенести вызов SeedTestData в LoadData(), где он вызывается один раз при старте с полным списком складов, и убрать его из LoadProducts().

Вывод

Работа с техническим заданием от преподавателя дала специфический опыт: нужно было не придумывать задачу, а читать и интерпретировать уже сформулированные требования. Это навык, который в реальной разработке применяется постоянно — техническое задание всегда пишет кто-то другой, и умение вычленивать из текста ключевые архитектурные решения дорогого стоит.

Первое, что я поняла: вдумчивый анализ задания до написания кода — это не потеря времени, а экономия его. Понимание того, что накладная должна иметь два состояния, что остаток меняется только после подтверждения, что удаление организации подтверждается вводом названия — всё это нужно было извлечь из текста задания до написания первой строки кода. В итоге архитектура сложилась без существенных переделок.

Второе: требование «каждый метод сервиса покрыть тестом» изменило подход к проектированию. Зная, что метод будет тестироваться, проектируешь его проще и предсказуемее: одна ответственность, чёткие входные условия, явный возврат результата вместо исключений там, где это уместно. Тесты стали не формальной повинностью, а инструментом проектирования.

Третье: разделение решения на три проекта дало неожиданный практический выигрыш. Тесты запускаются за секунды, потому что не тянут за собой WPF. Логика в библиотеке классов не может случайно обратиться к интерфейсу — компилятор этого не позволит. Такая дисциплина делает код чище.

Четвёртое: партионный учёт через накладные — интересная задача моделирования. Флаг состояния накладной решает сразу несколько проблем: он отделяет процесс формирования документа от процесса изменения остатков, позволяет атомарно проводить или откатывать изменения, и даёт возможность вести журнал проведённых операций. Это паттерн, который встречается в реальных учётных системах.

Пятое: столкновение с ошибками кодировки, XAML-парсинга и привязок данных показало, насколько важно уметь читать сообщения об

ошибках. Каждое исключение содержит конкретную подсказку — номер строки, имя ресурса, тип несовпадения. Научиться читать эти сообщения — важнее, чем запомнить синтаксис.

В целом работа над приложением дала опыт, который выходит за рамки одной дисциплины: проектирование многоуровневой архитектуры, покрытие тестами, работа с WPF и MVVM, обработка ошибок пользовательского ввода. Все эти навыки напрямую применимы в реальной разработке.